CodeDNA: An In-Source Communication Protocol for AI Coding Agents (v0.9)

Fabrizio Corpora

Independent Research · *fabrizio.corpora@gmail.com*

Zenodo preprint · April 2026 · Updated from doi:10.5281/zenodo.19158336 (v0.7, March 2026)

Abstract

LLM agents that autonomously edit production codebases suffer from *context blindness*: they modify a function without awareness of cross-file dependencies, producing silent bugs and broken cascades. Existing mitigations — RAG, project-level rules files (CLAUDE.md, AGENTS.md), or long-context injection — impose retrieval latency, maintenance overhead, or infrastructure cost. Recent work (ETH Zurich, 2026) shows that LLM-generated context files can *degrade* task success by 3% and increase inference costs by over 20%. We present **CodeDNA**, an in-source communication protocol that embeds dependency graphs, public contracts, agent constraints, and semantic naming conventions directly into source files. The standard is named after its biological parallel: just as DNA encodes the complete blueprint of an organism in every cell, CodeDNA ensures that every code fragment carries sufficient architectural context for an agent to act correctly in isolation. We evaluate CodeDNA v0.9 with an extended multi-model SWE-bench evaluation across **3 LLMs** (Gemini 2.5 Flash, DeepSeek Chat, Gemini 2.5 Pro) — **CodeDNA improves file localization F1 on all three**, with the strongest and most statistically robust result on **DeepSeek Chat**: **+17.1 percentage points** mean F1 gain across **10 Django tasks** (50.9% → 68.0%, Wilcoxon exact **p=0.0009765625**, wins/losses/ties = 10/0/0). 6 of the 10 DeepSeek tasks were independently replicated by an external contributor on a separate machine, strengthening replicability. Gemini 2.5 Flash shows +13pp (60% → 72%, p=0.040) on 5 tasks and Gemini 2.5 Pro shows +9pp (60% → 69%). Protocol v0.9 introduces three new fields — `related:` (cross-cutting semantic links), `wiki:` (opt-in pointer to curated deeper context), and `message:` (inter-agent conversational layer) — together with a tooling layer that turns annotations into a navigable Obsidian-ready knowledge vault. These results measure *navigation efficiency* (which files the agent reads), not end-to-end patch correctness, isolating the navigation bottleneck that CodeDNA specifically addresses. CodeDNA requires no build step, no external system, and zero retrieval latency.

1. Introduction

The adoption of LLM coding agents in production workflows has exposed a class of failure that single-file benchmarks routinely miss: *cross-file context blindness*. When an agent reads a function in isolation — because it operates in sliding-window mode or because the relevant contract resides in another module — it cannot observe the architectural constraints governing correct behaviour.

SWE-bench [1], the field's principal multi-file benchmark, demonstrated in 2024 that state-of-the-art models could resolve only 1.96% of real GitHub issues. The dominant failure mode was not reasoning quality but *navigation cost*: finding the right files required too many tool interactions. Subsequent work confirms the pattern: LoCoBench-Agent [2] (2025) measures tool-usage efficiency in long-context engineering workflows and finds systematic over-exploration; PRDBench [3] (2025) reports that agents spend the majority of their token budget on navigation rather than repair.

Current mitigation approaches share a structural weakness: *context lives outside the code*. CLAUDE.md and CursorRules are session-level files invisible to agents in sliding-window mode. RAG [6] systems introduce retrieval latency and require embedding infrastructure. A 2026 ETH Zurich study [4] found that LLM-generated context files (AGENTS.md) degraded task success by 3% and increased costs by over 20%.

We propose a complementary approach: embed context *inside the code*, at every level, so that any fragment an agent reads is already self-sufficient.

Contributions:

(1) The **CodeDNA Communication Protocol v0.9**: a four-level (Level 0–3), language-agnostic in-source communication protocol for AI coding agents, extending v0.7 with three new fields — `related:` (cross-cutting semantic links that bypass import graphs), `wiki:` (opt-in pointer to curated markdown deeper than the terse header), and `message:` (inter-agent conversational layer for observations not yet certain enough to become rules).

(2) A biological analogy that frames the standard's design principle and motivates its redundancy property.

(3) A **multi-model SWE-bench benchmark** across 3 LLMs with an extended DeepSeek evaluation (10 tasks, 3 runs each) demonstrating broadly beneficial improvements in file localization F1.

(4) Evidence of **+17.1pp mean F1 improvement** on DeepSeek Chat across 10 tasks (Wilcoxon exact p=0.0009765625, wins/losses/ties 10/0/0), **+13pp** on Gemini 2.5 Flash (p=0.040), and **+9pp** on Gemini 2.5 Pro — with 6 of the 10 DeepSeek tasks

independently replicated by an external contributor on a separate machine.

(5) A **tooling layer** (`codedna wiki bootstrap` and `codedna wiki sync`) that converts CodeDNA annotations into a navigable Obsidian-ready knowledge vault and a Karpathy-style narrative project wiki — turning in-source annotations into a human-browsable graph.

(6) Open-source release of spec, benchmark code, and AI integration files.

2. Related Work

2.1 Multi-File Agent Benchmarks

SWE-bench [1] (Jimenez et al., 2024) evaluates LLMs on 2,294 real GitHub issues across 12 Python repositories. It established that multi-file coordination is the principal bottleneck, with a 1.96% resolution rate at launch. CodeDNA addresses the *navigation cost* dimension that SWE-bench exposed.

LoCoBench-Agent [2] (November 2025) provides the closest methodological parallel to our work: it benchmarks LLM agents in long-context software engineering scenarios with 8,000 tasks and measures *tool-usage efficiency* explicitly. Our metrics (tool calls per bug, file reads, grep frequency) directly parallel LoCoBench-Agent's efficiency framing.

PRDBench [3] (October 2025, presented at AAMAS 2026) uses 50 real Python projects and an Agent-as-a-Judge paradigm to evaluate agents from Product Requirement Document to implementation. It confirms that agents over-explore when codebase structure is opaque.

HumanEval [5] (Chen et al., 2021) measures single-function synthesis from docstrings. Its *pass@k* metric is complementary: it captures output quality but not cross-file navigation, making it orthogonal rather than competing.

2.2 Context Management for LLM Agents

Retrieval-Augmented Generation [6] retrieves relevant chunks from a vector store at inference time. Applied to code, retrieval quality depends on embedding similarity, which may not align with architectural relevance and requires additional infrastructure.

Rules files (CLAUDE.md [10], CursorRules, AGENTS.md, GitHub Copilot instructions [9]) impose project-level conventions but are invisible to sliding-window agents and require manual maintenance. The ETH Zurich study [4] provides the strongest evidence against the external-file approach: even carefully crafted human-written files offer only a 4% gain, and LLM-generated ones actively degrade performance by −3% while increasing costs by +20%.

MemGPT [7] maintains compressed memory across turns. This addresses recall of prior context but not forward navigation: a history of prior reads does not tell an agent what to read *next*. Knowledge graph–based approaches (Tongyi Lingma [11], 2025–2026) use graph traversal to expose cross-module call relationships but rely on external infrastructure and dynamic analysis. **CodeDNA's key differentiator**: the navigation graph is co-located with the code itself, requiring no retrieval system and surviving in any sliding window.

2.3 Code Annotation and Documentation Standards

Documentation standards (docstrings, JSDoc, Doxygen, Literate Programming [8]) focus on human readability and API documentation generation. They do not encode cross-file navigation semantics or agent-specific constraints. Type annotation systems (PEP 484, TypeScript) encode type contracts but not architectural ones. CodeDNA differs by designing annotations as a *machine-readable navigation protocol* targeting the failure modes of LLM agents rather than human readers.

3. The CodeDNA Communication Protocol v0.9

3.1 Biological Analogy

The name *CodeDNA* captures the standard's core design principle by analogy to molecular biology. Biological DNA encodes the complete blueprint of an organism in *every cell*: each cell's nucleus contains the full genome, not just the instructions for its own specialised function. This redundancy means that any cell — regardless of its location or context — can be read by the right machinery and yield architectural information about the whole organism.

CodeDNA applies the same principle to software: every source file encodes not only its own logic but also its *position in the architectural organism* — what it depends on, what depends on it, what contracts it must honour. The protocol comprises four levels (0–3): Level 0 is the project manifest (`.codedna`), operating at repository scale. Levels 1–3 operate at file, function, and variable scale respectively, and mirror three scales of biological encoding:

Table A. Biological DNA ↔ CodeDNA Analogy

Biological concept	CodeDNA equivalent
Full genome in every cell	L1 module header in every file: full architectural position
Gene = local function unit	L2 function docstring: local contract + cross-file deps
Codon = minimal encoding unit	L3 inline comment at call site: 1-line context at danger point
Holographic redundancy	Any 10-line extract carries enough context to navigate safely
No external lookup needed	Zero retrieval latency; context travels with the code

The holographic property — cutting a hologram in half produces two complete smaller images, not two halves — explains CodeDNA's sliding-window robustness: even when an LLM reads only lines 50–80 of a file, it encounters Level 2 function docstrings and Level 3 call-site comments that restate the critical architectural rules at the exact point of risk.

3.2 Level 0 — Project Manifest

A single YAML file (`.codedna`) at the repository root. The agent reads this first — before opening any source file — to understand package purposes and inter-package dependencies. It is

injected into the agent's initial context so the full structural map is available before any tool call is made.

```
project: myapp
packages:
  payments/:
     purpose: "Invoice generation, payment process-
ing"
  analytics/:
    purpose: "Revenue reports, KPI dashboards"
    depends_on: [payments/, tenants/]
  tenants/:
    purpose: "Multi-tenant management, suspension"
```

The manifest is the entry point for the **Planner Read Protocol**: (1) read `.codedna`; (2) read only the module docstring of each candidate file (first 8–12 lines); (3) build an `exports:` → `used_by:` graph; (4) open only files where the graph intersects with the task. This pattern reduces unnecessary reads while maximising coverage of architecturally relevant files.

3.3 Level 1 — Module Header

A Python-native module docstring that must be the first content in every file. It encodes the full architectural position of the file in approximately 70 tokens.

```
"""analytics/revenue.py — Monthly revenue aggrega-
tion.

deps:     payments/models.py → get_invoices_for_pe-
riod
         tenants/models.py → is_suspended
exports: monthly_revenue(year,month)->dict
used_by: api/reports.py → revenue_route
tables:  invoices(tenant_id, amount_cents, status)
         tenants(suspended_at, deleted_at)
rules:   get_invoices_for_period() returns ALL ten-
ants,
           NO suspended filter → MUST call is_sus-
pended()
         BEFORE aggregating revenue
"""
```

Using Python's native triple-quote string format (rather than a custom comment block) maximises LLM comprehension: models trained on billions of Python files apply existing pattern-matching to the docstring's structure, rather than parsing an unfamiliar protocol from scratch.

The `rules` field encodes hard constraints that apply to every edit in the file. Unlike inline annotations (function-scoped), `rules` applies globally, acting as the cell's genome: available anywhere in the file without requiring the reader to navigate to a specific function.

3.4 Level 2 — Sliding-Window Annotations

Agents frequently extract 50–100 lines from the middle of a file, bypassing the module header. Level 2 embeds navigation cues at the function level via two sub-layers.

Level 2a — Function Docstring (Google style):

```
def monthly_revenue(year: int, month: int) -> dict:
    """Aggregate paid invoices into monthly totals.

     Depends: payments.models.get_invoices_for_peri-
od
               — returns ALL invoices, NO suspended
filter.
    Rules:   MUST filter is_suspended() BEFORE sum-
ming.
    """
```

Level 2b — Call-site inline comment (last line of defence):

```
    # includes suspended tenants — filter below
    raw = get_invoices_for_period(year, month)
    total = sum(i['amount_cents'] for i in raw)
```

The call-site comment ensures that even a 10-line window centred on the dangerous call carries the critical context. This is the codon-level encoding: the smallest unit that still conveys architectural meaning.

3.5 Level 3 — Semantic Naming

Variable names encode type, shape, domain, and origin. Format: `<type>_<shape>_<domain>_<origin>`.

```
# ❌ Standard — agent must trace to understand
data  = get_users()
price = request.json["price"]

# ✅ CodeDNA — self-documenting in any 10-line win-
dow
list_dict_users_from_db  = get_users()
int_cents_price_from_req = request.json["price"]
```

3.6 The `related:` field (v0.9)

The `used_by:` graph captures *structural* dependencies — who imports whom. Many real bugs, however, span files that *share a pattern* without any import link: IDNA encoding helpers scattered across `validators`, `encoding`, `html`, and `mail.utils`; timezone handling duplicated across backend modules; equality-check idioms repeated in unrelated classes. For these *cross-cutting concerns*, no chain of `used_by:` edges connects the files the agent must edit. The v0.7 protocol showed Δ≈0% on this class of task (see §6.5).

Version 0.9 introduces the optional `related:` field — a *semantic* link captured alongside the structural one:

```
"""mail/utils.py — email helpers.

exports: class CachedDnsName | DNS_NAME
used_by: mail/message.py → DNS_NAME
related:  core/validators.py — shares  IDNA/punycode
logic
         utils/encoding.py — encoding utilities for
                             non-ASCII domains
rules:   get_fqdn() returns raw unicode — callers
         must handle non-ASCII
"""
```

Empirically, on Django bug #11532 (unicode domain crash, fix spans 5 files with no shared import ancestor): a CodeDNA annotation using only `used_by:` finds 2 of 5 files (F1=40%, matching the control baseline). Adding `related:` brings the agent to **5 of 5** files (F1=100%). The semantic link closes the

cross-cutting gap that was identified as the structural limit of v0.7.

3.7 The `wiki:` opt-in field and the knowledge-vault layer (v0.9)

A single-line `rules:` entry cannot carry narrative context — decision history, extension points, subtle invariants that require a paragraph. Dumping everything into the docstring bloats the module header the sliding window must traverse. v0.9 adds the optional `wiki:` field, a pointer to a curated markdown page stored under `docs/wiki/`:

```
"""cli.py — CodeDNA annotation tool.

exports: scan_file | run
used_by: tests/test_cli.py → FileInfo
wiki:    docs/wiki/cli.md       ← opt-in
rules:   ...
"""
```

Sparsity is the signal. The agent workflow is: if `wiki:` is present, read the pointed markdown first; if absent, the docstring is sufficient. A prior agent left the pointer only when they decided the terse header was insufficient. Unlike a blanket "compile a wiki for every file" approach, this preserves the "every fragment carries the whole" principle at the cheap default and opts into deeper context only where warranted.

The companion tooling — `codedna wiki bootstrap` and `codedna wiki sync` — converts the annotation graph into two complementary artifacts. **Bootstrap** emits a nested Obsidian vault (one markdown page per source file, mirroring the source tree, with `[[wikilinks]]` derived from `used_by:` and `related:`) — a human-browsable graph with no additional annotation effort. **Sync** regenerates a single narrative project wiki (`docs/codedna-wiki.md`) following a seven-section template (identity, topology, workflows, hotspots, and the refresh protocol) — designed for agents entering a repository cold. Both are deterministic and idempotent; wired to a post-commit hook they stay current without relying on an agent to remember to update them.

3.8 The `message:` inter-agent channel (v0.9)

Every `agent:` entry may carry a `message:` sub-field — a conversational layer for observations not yet certain enough to become `rules:`, open questions, or soft handoff notes for the next agent. The field lives at two levels: inside module headers (Level 1) for agents reading the whole file, and inside function docstrings (Level 2) for agents operating in sliding-window mode that never see the header.

```
agent:   claude-sonnet-4-6 | 2026-03-10 | impl X
         message: "rounding edge case in multi-
                  currency — not yet sure if this
                  should be a rule"

agent:   gemini-2.5-pro | 2026-03-18 | added Y
         message: "@prev: confirmed → promoted
                  to rules:"
```

A `message:` is append-only — a later agent either *promotes* it to `rules:` (with `@prev: promoted to rules:`) or *dismisses* it with an explicit reason (`@prev: not applicable because...`). Both outcomes preserve the full audit trail. In the AgentHub multi-agent SaaS experiment (DeepSeek R1, 5 agents, 83 minutes), `message:` was adopted spontaneously in 54 out of 55 annotated files (**98.2%**) with no mid-session reminders — agents used it for handoff notes, per-function gap annotations, and cross-file constraint propagation.

4. Benchmark Design

We measure *navigation efficiency* — how many tool interactions an autonomous agent requires to correctly localize the root cause of a bug in a multi-file codebase.

4.1 Agent Implementation

We implemented an autonomous agent using Gemini 2.5 Flash (temperature 0) with three function-calling tools: `read_file(path)`, `list_files(directory)`, and `grep(pattern, directory)`. No file contents are pre-loaded. The agent runs to completion (max 30 turns) or until it stops calling tools. While temperature 0 minimises sampling variance, we note that the full agent loop (tool orchestration, API behaviour) is not strictly deterministic in the mathematical sense. Metrics: **total tool calls**, **read_file calls**, **grep calls**, **conversation turns**, and **found_correct** (boolean — whether the identified root-cause file is architecturally correct).

5. Results

5.1 SWE-bench Multi-Model Evaluation

To test whether CodeDNA's benefit generalises across different LLM architectures, we evaluated 3 models on real Django issues from SWE-bench. Each model received identical tools (`read_file`, `list_files`, `grep`), the same system prompt, and a maximum of 30 turns. Each condition was repeated 3–5 times per task at T=0.1 to obtain stable mean estimates; statistical significance is evaluated with the Wilcoxon signed-rank test (one-tailed, H1: CodeDNA > Control) over the per-task mean F1 pairs. The primary metric is **file localization F1**: the harmonic mean of precision (fraction of files read that are in the ground-truth patch) and recall (fraction of ground-truth files that the agent actually read). We emphasise that this metric measures *navigation accuracy* — whether the agent reads the right files — not whether it produces a correct patch. Isolating the navigation component allows us to evaluate CodeDNA's specific value proposition without conflating it with the model's code synthesis ability. The DeepSeek condition was extended in April 2026 from 5 to **10 tasks**, with 6 of the new tasks independently executed by an external contributor on a separate machine (see §5.1.1).

Table 3. *Multi-model file localization F1, Control vs. CodeDNA. DeepSeek covers 10 tasks; Gemini Flash and Pro cover 5 tasks each. 3–5 runs per task at T=0.1.*

Model	Tasks	Ctrl F1	DNA F1	Δ F1	p-value	Tasks Won
DeepSeek Chat	10	50.9%	68.0%	**+17.1pp**	0.001**	10 / 10
Gemini 2.5 Flash	5	60%	72%	**+13pp**	0.040*	4 / 5
Gemini 2.5 Pro	5	60%	69%	**+9pp**	0.11	3 / 5

Key result: DeepSeek Chat (10 tasks, W+=55, N=10) achieves strong statistical significance at **p=0.0009765625** (Wilcoxon exact, one-tailed), with CodeDNA winning on all 10 tasks — no inversions, no ties. Gemini 2.5 Flash (5 tasks, W+=14, N=5) achieves p=0.040 (Δ=+13pp). Gemini 2.5 Pro (5 tasks, W+=12, N=5) shows Δ=+9pp at p=0.11. Task 13495 retains its model-dependent anomaly — Flash benefits strongly (+22pp) while Pro regresses (−8pp) — but under the expanded DeepSeek condition Δ on 13495 is now positive (+6.6pp), suggesting the prior negative reading reflected sampling noise on a small N rather than a structural regression. Per-task breakdowns in the repository.

5.1.1 Independent replication on DeepSeek Chat

Between the v0.7 submission and this revision, an external contributor (@fabioscialanga) independently executed 6 additional DeepSeek tasks (13121, 15629, 16263, 11400, 11883, 11808) on a separate machine, using the same runner, the same prompts, and the same task definitions as the original 4 tasks. The raw session traces were merged into the repository (`labs/benchmark/runs/deepseek-chat/ session_traces/`) and the combined 10-task analysis regenerated with the repository's analysis script. The result is a single coherent dataset with transparent provenance: every session trace carries the originating machine's git commit and run manifest. All 10 tasks favour CodeDNA. Two tasks (11808 and 13121) exhibit a notable stability pattern — the standard deviation of F1 across three CodeDNA runs is **0.00** (the agent produces the same file-localisation decision every time), while the control standard deviation is 0.20–0.25. The advantage is not merely a higher mean; it is a reduction in the run-to-run unpredictability that production deployments care about. Independent replication on heterogeneous hardware with matching protocol is the strongest evidence the current sample size permits; it meaningfully raises confidence that the DeepSeek effect is not an artefact of a single machine or a single rollout of the sampling loop.

Under the v0.9 dataset, **DeepSeek Chat shows both the largest effect (+17.1pp) and the lowest p-value (0.001)**, while Gemini 2.5 Flash (+13pp, p=0.040) and Gemini 2.5 Pro (+9pp, p=0.11) remain positive on 5 tasks each. Gemini 2.5 Pro ctrl=60% matches Flash, suggesting these tasks do not favour larger models in the control condition. The three models use different provider stacks (Google native function calling vs. DeepSeek's OpenAI-compatible endpoint), which introduces some cross-provider variability not fully controlled. This does not invalidate the comparison — the annotations themselves are identical — but the observed differences reflect both CodeDNA's effect and provider-specific runtime behaviour. The pattern that cheaper or less capable models benefit most from CodeDNA — discussed below — is consistent with DeepSeek (the cheapest of the three by cost-per-token) showing the strongest statistically validated gain.

5.2 When CodeDNA Helps Most

Analysis by task type reveals how the benefit depends on structural properties. The first four rows report `used_by:`-only results on Gemini 2.5 Flash (5 tasks, ≥5 runs each, v0.7 protocol). The cross-cutting row (11808) reports both the original v0.7 baseline and the v0.9 result with the `related:` field (§3.6):

Table 3a. *Δ F1 by task type. Rows 1–4 on Gemini 2.5 Flash (v0.7 protocol). Row 5 reports v0.7 baseline, v0.9 with `related:`, and the 10-task DeepSeek result.*

Task type	Task	Δ F1
Clear dependency chain — A calls B delegates to C	12508 (dbshell)	**+9%**
Delegation with backend fan-out — one interface, N backends	13495 (Truncatzinfo)	**+22%**
Feature addition with flag gating — across feature/schema layers	11991 (INCLUDE)	**+17%**
XOR feature with multi-layer propagation	14480 (Q() XOR)	**+17%**
Cross-cutting fix — same pattern in N independent files, no shared ancestor	11808 (__eq__) · 11532 (IDNA domains)	v0.7: ≈0% · **v0.9 `related:`: 11532 F1 40%→100%; 11808 Δ=+25.5pp (DeepSeek 10-task)**

CodeDNA with `used_by:` alone is most effective when there is a navigable call chain. For cross-cutting concerns — the same fix in N independent files with no common ancestor — the benefit of `used_by:` alone is near-zero because no navigation path connects them. This was the structural limit of v0.7, reported transparently rather than hidden. **v0.9 addresses this class of bug through the `related:` field (§3.6):** on Django bug #11532 (IDNA domain crash, 5-file cross-cutting fix) F1 jumps from 40% to 100% when `related:` is added; on task 11808 under the 10-task DeepSeek condition, Δ=+25.5pp (with the caveat that this confounds `related:` with model differences — a clean ablation remains open). See §5.3 for the annotator integrity audit and §6.6 for the proposed project-level `cross_cutting_patterns:` manifest extension that complements the file-level `related:`.

Per-task qualitative analysis. The five tasks span a spectrum of structural difficulty. **Task 12508** (dbshell) is the easiest to enter: the management command file is named after the feature; the challenge is discovering the four backend client files, which `used_by:` handles reliably (+9pp). **Task 11991** (INCLUDE) follows a similar pattern — `schema.py` is findable by grep, and `used_by:` enumerates all four backend schema editors explicitly (+17pp). **Task 14480** (Q() XOR) requires following a multi-layer chain — from `query_utils.py` through `where.py`, `sql/__init__.py`, and per-backend `features.py` — exactly

the cascade that `used_by:` enables (+17–18pp). **Task 13495** (Trunc tzinfo) is the most model-divergent: Flash gains +22pp while DeepSeek and Pro each regress by 8–9pp. The entry file (`db/models/functions/datetime.py`) is reachable for all models, but following `used_by:` chains to backend files appears to introduce a precision trap for DeepSeek and Pro — a structural anomaly under investigation. **Task 11808** (`__eq__`) was identified in v0.7 as the structural limit of the protocol: the entry point `models/base.py` is an 847-line generic file and the five affected subclasses share no common caller. v0.9 addresses this class of bug via the `related:` field (§3.6), which lets a file point to its semantic siblings outside the import graph. Under the expanded DeepSeek condition on 10 tasks, 11808 now shows Δ=+25.5pp — but this improvement mixes the effect of `related:` with model differences. A clean ablation (with/without the `related:` field on the same model) remains open future work.

One observation from the multi-model data is that cheaper or less capable models appear to benefit more from CodeDNA than frontier models. The v0.7 evidence for this was weak — DeepSeek and Pro showed non-significant Δ (p=0.11). The expanded DeepSeek condition (10 tasks, p=0.001, 10/0/0) now provides the first strong statistical support for this working hypothesis.

5.3 Annotation Integrity Audit

A key concern for any navigation benchmark is whether annotations guide the agent to the right files by describing the architecture, or by encoding the solution. We performed a full audit of the benchmark annotations to verify no task-specific hints were embedded.

Methodology. For each task, every ground-truth file (from `files_in_patch`) was inspected. The `used_by:` targets were classified as GT (ground-truth) or non-GT. The `rules:` field was evaluated for task-specific prescriptions vs. architectural descriptions. The test: *if an agent reads only the rules: field, does it still need to open files to determine what to fix?* A correct annotation passes this test; one that pre-populates a file list fails.

Table 3b. *Annotation integrity verdicts per task.*

Task	`used_by:` verdict	`rules:` verdict
11808 (`__eq__`)	Files independent — no `used_by:` chain by design (v0.7); v0.9 `related:` links the semantic siblings	Describes Python data model convention, not a file list. v0.7 Δ≈0% confirmed no `used_by:` advantage; v0.9 + 10-task DeepSeek shows Δ=+25.5pp
14480 (XOR)	16 targets in `query_utils.py`; only 2 are GT, both genuine callers	Describes connector system mechanics; agent must still reason which files are relevant
13495 (Trunc tzinfo)	`base/operations.py` lists all 4 backends — all backends that exist	`_convert_field_to_tz()` per-backend: accurate architecture, not a fix prescription
11991 (INCLUDE)	Corrected during audit to include all 4 backend schema editors (see note)	Architectural delegation pattern; no file list for the fix
12508 (dbshell)	`base/client.py` lists all 4 backend clients — complete by definition	Describes existing architecture accurately; no task-specific prescriptions possible

Annotation correction made during audit. `django/db/backends/base/schema.py` in task 11991 had an incomplete `used_by:` listing only `postgresql/schema.py`. All four backend schema editors (`mysql`, `oracle`, `postgresql`, `sqlite3`) genuinely inherit from `BaseDatabaseSchemaEditor`. The annotation was corrected to reflect the complete inheritance graph before benchmark runs.

Verdict. The benchmark annotations are architecturally accurate and do not encode task-specific solutions. Where GT files appear in `used_by:` targets, it is because those files are genuine callers or subclasses — not because the annotator cherry-picked them. The empirical results are consistent with this: tasks with navigable chains show Δ>0%; task 11808 showed Δ≈0% under v0.7 (no `used_by:` chain connecting the independent files) and improves to Δ=+25.5pp under v0.9 with the `related:` field closing the cross-cutting gap.

5.4 Retry Cost Analysis

File-localization F1 is typically reported as an efficiency metric. We argue it should also be treated as a *correctness proxy*: an agent that reads fewer than half the ground-truth files is unlikely to produce a complete patch, forcing the user to re-invoke the entire session. We define a run as *high-risk* when per-run recall falls below 50%.

Across 131 multi-run trials from the v0.7 five-task baseline (DeepSeek-Chat, Gemini 2.5 Flash, Gemini 2.5 Pro — 5 tasks each, 3–6 runs per task), the fraction of high-risk runs was:

Table 4. *Incomplete-fix risk (recall < 50%) pooled across three models and the five common SWE-bench tasks (v0.7 baseline).*

Condition	High-risk runs	Total runs	Risk rate
Control	34	66	**52%**
CodeDNA	16	65	**25%**
Reduction			**−27 pp (−52% relative)**

The most pronounced case from the v0.7 baseline was DeepSeek-Chat on task 11808 (10 ground-truth files): all five control runs achieved recall of 10–20% (finding 1–2 of 10 files); all five CodeDNA runs achieved 30–60% recall. Every control run would have produced an incomplete patch. Under CodeDNA, three of five runs exceeded the 50% threshold. The v0.9 expansion and the introduction of `related:` push this further — DeepSeek on 11808 in the 10-task condition shows Δ=+25.5pp F1 with std=0.00 across the three CodeDNA runs.

Gemini 2.5 Pro provides a complementary data point: CodeDNA reduced mean tool calls by 14.3% while raising mean F1 by 9 pp, demonstrating that structured context enables the model to navigate *directly* to relevant files rather than exploring broadly — fewer calls, higher accuracy.

These results support an *accuracy-first* framing for annotation protocols: even if CodeDNA were cost-neutral in tool calls, halving the rate of incomplete navigation would justify its adoption by halving the rate of sessions that must be repeated.

6. Discussion

6.1 Positioning Against Related Work

Table 5 situates CodeDNA within the landscape of related approaches:

Table 5. *Problem–Solution mapping of related work and CodeDNA.*

Work	Problem exposed	CodeDNA response
SWE-bench [1]	Multi-file navigation is the bottleneck (1.96% resolution)	Reduces navigation cost via embedded dependency graph
LoCoBench [2]	Tool-usage inefficiency in long-context workflows	Reduces tool calls on dependency-chain tasks (−14% on Gemini 2.5 Pro with +9pp F1)
PRDBench [3]	Over-exploration on opaque codebases	Transparent codebase structure eliminates aimless exploration
ETH Zurich [4]	External context files degrade performance −3%, cost +20%	Embedded context travels with code; no separate file to misuse
MemGPT [7]	LLMs forget prior context across turns	No memory needed: context is in the code the agent reads
RAG [6]	Retrieval latency and embedding infrastructure required	Zero retrieval: context is already at read time

6.2 Why In-Source Annotation Works

The mechanism mirrors the biological insight: when the contract governing a function is declared *at the function*, capable agents do not need to retrieve it from its original definition. When a `rules` field propagates a contract downstream to the exact location where it is violated, the agent can diagnose the issue correctly without following the dependency back to its original definition.

The ETH Zurich finding [4] that LLM-generated context files *degrade* performance precisely motivates this approach: if external context files cause over-exploration, the solution is not better files but embedded context that arrives *with the code the agent was already going to read*. CodeDNA turns every file read into a context delivery event.

6.3 Format Design Rationale

The choice of Python-native module docstrings (rather than custom comment blocks) for Level 1 was validated empirically: models trained on Python corpora apply existing pattern-matching to the docstring's `deps:` / `exports:` / `rules:` structure. The custom `# === CODEDNA:0.5 ===` block format, while machine-parsable, required the model to process an unfamiliar syntax schema, yielding marginally longer reasoning chains before exploitation.

6.4 Annotation Overhead and the Agentic Workflow Framing

A common objection to annotation standards is the overhead of writing and maintaining them. For CodeDNA, this objection must be evaluated in context. CodeDNA is designed for *agentic code generation workflows* — environments in which an LLM agent both generates and modifies source files. In this setting, the agent itself produces the module headers and function docstrings as a natural byproduct of generation. The human developer does not annotate 500 files manually; the coding agent leaves its architectural memory in-place as it writes. The marginal cost of annotation approaches zero.

This framing also redefines what CodeDNA *is*: not a standard that asks developers to write more comments, but a convention that gives agents a structured format in which to record their own local architectural understanding. The standard converts agent-generated code from a passive output into an active, self-describing navigation graph for the next agent that reads it.

A secondary observation is that CodeDNA's structured fields (`deps:`, `rules:`, `Depends:`) do not require new syntax — they live inside Python-native docstrings. Many codebases already have docstrings; CodeDNA adds *navigation intent* to documentation that already exists. The distinction between a standard docstring and a CodeDNA docstring is not syntactic but semantic: the latter must be *machine-oriented and navigation-oriented*, not merely human-readable.

6.5 Limitations

Sample size. The v0.9 SWE-bench evaluation uses 3–5 runs per task at T=0.1, enabling Wilcoxon significance testing. DeepSeek covers N=10 tasks (p=0.001); Gemini Flash and Pro cover N=5 each. All tasks are drawn from a single Django repository, which limits generalisability. Future work should expand Flash and Pro to N=10, add a second repository, and evaluate on the full SWE-bench suite with real-agent toolchains (Devin-style, Cursor, GPT-tools loop).

Localization vs. resolution. The SWE-bench evaluation measures file localization F1, not end-to-end patch correctness.

While localization is a necessary precondition for correct patching, it is a proxy metric. Whether CodeDNA's navigation improvements translate to higher resolution rates remains an open question for future work.

Annotation synchronization. The primary maintenance challenge is not *writing* annotations (in agentic workflows the cost is near-zero) but ensuring they remain *consistent* with the code over time. If an agent modifies a function without updating the enclosing `rules:` or `Depends:` fields, the annotation becomes misleading — potentially worse than no annotation. This shifts the research problem from production cost to *consistency enforcement*: pre-commit hooks, CI validation, and agent-side re-annotation obligations are the correct mitigations. A complementary direction is a "CodeDNA Bootstrapper" — a batch LLM pass that injects initial annotations into legacy codebases, preparing them for agentic maintenance. Future integrations should treat annotation drift as a first-class failure mode.

Multi-model evaluation. All 3 models complete. Results are consistent across models — positive Δ on 10/10 DeepSeek tasks, 4/5 Flash, 3/5 Pro — but the task set is still Django-only. The task 13495 anomaly from v0.7 (Flash +22pp, Pro/DeepSeek negative) remains partly visible on Gemini Pro (−8pp) but has flipped positive on DeepSeek under the expanded 10-task condition (+6.6pp), suggesting the prior negative reading reflected small-N sampling noise rather than a structural regression. Cross-provider runtime differences (function-calling implementations, tool orchestration) introduce variability that is not fully controlled. The results should be replicated on a larger, more diverse task set before generalisability claims can be strengthened.

Bug complexity. The SWE-bench tasks evaluated each require understanding one or two cross-file contracts. Bugs spanning five or more architectural layers may amplify the advantage further — a hypothesis that requires a larger-scale evaluation to test.

Results may represent a lower bound, pending empirical validation. The v0.9 benchmark isolates the effect of the annotations themselves under a zero-shot evaluation — the models have no prior training on the CodeDNA format. A fine-tuned model might treat `used_by:`, `related:`, and `rules:` as native navigation signals, but this has not been tested. A second open question is whether the `message:` conversational layer reduces retry rate across sessions: within a single session it was spontaneously adopted at 98.2% in the AgentHub experiment, but a longitudinal benchmark across many sessions of the same codebase is not yet available. A third open question is whether project-level `cross_cutting_patterns:` (§6.6) further amplifies the `related:` gain. The current results may therefore represent a lower bound, but this framing itself requires validation.

6.6 Future Work

Extended benchmark coverage. The v0.9 evaluation covers 20 per-model-task F1 pairs across 3 LLMs (10 DeepSeek + 5 Flash + 5 Pro). Priority directions include expanding Flash and Pro to 10 tasks each to match the DeepSeek sample size, adding a placebo condition (a CodeDNA-looking header populated with plausible-but-false annotations) to isolate the *correctness* of the annotations from the mere presence of structured text in the header, and end-to-end patch-correctness metrics that complement navigation F1 (which this paper measures in isolation by design).

LLM-assisted generation of `related:`. The `related:` field closes the cross-cutting gap empirically (§3.6, §6.5), but population today is manual or rule-based. A proposed extension uses a single LLM pass per file to suggest semantic links from the full corpus — detecting shared patterns across unrelated files — with the agent retaining veto authority. The same mechanism can produce the `cross_cutting_patterns:` field discussed below.

Agent telemetry via git trailers. Git is already immutable, append-only, and diff-complete. A proposed extension embeds AI session metadata directly in commit messages using *git trailers* — the same convention as `Co-Authored-By:`, natively recognised by GitHub and standard git tooling. The proposed fields are: `AI-Agent:`, `AI-Provider:`, `AI-Session:`, `AI-Visited:` (files read during the session, which git does not track natively), and `AI-Message:`. Combined with the existing `.codedna` session log and the file-level `agent:` field, this creates a *three-tier audit architecture* linked by a shared `session_id`: git (authoritative diff and metadata), `.codedna` (navigable session summary for agent planning), and file `agent:` (sliding-window-safe one-liner). The architecture enables immediate audit queries without external tooling — for example, retrieving all files an agent read in a given session, or comparing navigation behaviour across model providers over the project history.

Project-level cross-cutting patterns (`cross_cutting_patterns:`). The `related:` field (§3.6) addresses cross-cutting concerns at the *file level*: each file individually points to its semantic siblings. A complementary direction is a Level 0 extension that adds a `cross_cutting_patterns:` field to the `.codedna` manifest, populated by an agent *post-fix* as accumulated project-wide knowledge: *"all classes implementing `__eq__` must return `NotImplemented` for unknown types."* This is architecturally distinct from pre-populating a file list for evaluation — the pattern is written by an agent that has already navigated the codebase. Under the expanded DeepSeek condition, task 11808 (the cross-cutting `__eq__` example) now shows Δ=+25.5pp — but this improvement is confounded with model differences, so a clean ablation on the contribution of `related:` alone remains open.

Verification agents. The annotation synchronization risk identified in §6.5 — annotations diverging from the code they describe — motivates a class of *verification agents* that periodically cross-check annotations against the actual codebase. Because annotations are structured and machine-readable, this maintenance is itself automatable. Proposed verification types include: navigation correctness (does following `used_by:` links reach the correct files?), rule compliance (does the code respect its own `rules:` constraints?), annotation accuracy (do `exports:` symbols match the actual public API?), and `message:` lifecycle (are open observations stale or resolved?). Verification

agents represent the natural complement to annotation-writing agents — closing the feedback loop between the protocol and the code it describes.

CodeDNA as a self-generating training corpus. The following section describes a speculative future direction that has not yet been empirically validated. A consequential but under-explored property of the protocol is that a CodeDNA-annotated codebase with active agent usage constitutes a *self-generating training corpus* across three complementary levels of supervision signal.

At the *supervised fine-tuning* (SFT) level, the append-only `agent:` field accumulates an ordered, file-scoped narrative of correct navigational decisions. Each line encodes — in natural language — what an agent did, what constraint it discovered, and what it left for the next session. Across thousands of files and sessions, this constitutes a dense demonstration dataset of expert codebase navigation grounded in real task outcomes.

At the *preference alignment* (DPO/RLHF) level, the git history with AI trailers provides naturally occurring *(rejected, chosen)* pairs. When Agent A introduces an incorrect annotation or violates a domain constraint, and Agent B subsequently corrects it and updates `rules:`, the two commits form a labeled preference pair — complete with visited-file lists, session identifiers, and agent-written rationale in `AI-Message:`. Unlike synthetic preference data, these pairs are grounded in the actual codebase state at the time of each decision, carry the full navigational context via `AI-Visited:`, and are produced at zero marginal cost during normal development.

At the *process reward* (PRM) level, the session trace infrastructure already present in the benchmark runner (`traces_to_training.py`) records the full ordered sequence of tool calls per session. Combined with the binary task outcome (patch applied successfully or not), each trace becomes a labeled reasoning trajectory. Steps that led to a correct outcome receive positive reward; steps on a session that failed can be assigned negative signal, enabling a process-level reward model trained on real agent behaviour rather than on human-generated chain-of-thought.

These three levels are mutually reinforcing. Models fine-tuned on CodeDNA-derived SFT data navigate annotated codebases more accurately, producing higher-quality annotations; higher-quality annotations yield more informative preference pairs; more informative preference pairs improve process reward models that further sharpen navigational policy. This constitutes a *data flywheel*: each agent session that writes or corrects a CodeDNA annotation improves the training signal available to all future model generations — without requiring additional human labelling effort or dedicated data collection infrastructure. The codebase is simultaneously the environment, the protocol, and the training dataset.

7. Conclusion

We presented CodeDNA, a four-level (Level 0–3) in-source communication protocol whose design uses a biological analogy as a guiding intuition: just as DNA encodes a complete organism blueprint in every cell, CodeDNA ensures that every source file — and every function within it — carries enough architectural context for an LLM agent to navigate correctly without external lookup.

Our SWE-bench multi-model evaluation provides evidence that in-source architectural context reduces navigation cost for LLM agents. The strongest result is on **DeepSeek Chat**: **+17.1pp** mean F1 gain across **10 Django tasks** (50.9% → 68.0%, Wilcoxon exact p=0.0009765625, 10/0/0) — with 6 of the 10 tasks independently replicated by an external contributor. Gemini 2.5 Flash shows +13pp (60% → 72%, p=0.040) on 5 tasks and Gemini 2.5 Pro shows +9pp (60% → 69%), with positive Δ on 4/5 and 3/5 tasks respectively. These results suggest CodeDNA functions as a *navigation prior* that benefits diverse model architectures; the sample size is strongest on DeepSeek (N=10) and remains smaller on Flash and Pro (N=5), all drawn from a single Django repository.

The results are consistent with the core hypothesis: *when cross-file contracts are declared at the point of use, agents can navigate more efficiently without following each contract back to its source.* Whether this navigation improvement translates to higher end-to-end bug resolution rates is an important open question that we leave to future work with larger-scale evaluations.

Fine-tuning potential and improvement ceiling. All benchmark results reported here are *zero-shot*: models follow `used_by:` links and `rules:` constraints by general language understanding alone, with no prior training on the CodeDNA protocol. Whether fine-tuning would further improve results is an open empirical question. v0.9 introduces three features — `related:`, `wiki:`, and `message:` — that address the three most consequential limitations of v0.7 (cross-cutting concerns, narrative context beyond the header, and inter-agent memory). Remaining open directions include: (1) fine-tuning on CodeDNA-annotated corpora so models treat structured fields as native navigation signals rather than free text; (2) a project-level `cross_cutting_patterns:` manifest complementing the file-level `related:`; (3) a longitudinal `message:` benchmark across many sessions on the same codebase. Whether any of these yields further gains is an open empirical question.

The specification (v0.9), benchmark code, raw session traces (including the independently replicated DeepSeek runs), and AI integration files are available at:

github.com/Larens94/codedna

References

[1] Jimenez, C., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

[2] LoCoBench-Agent: *An Interactive Benchmark for LLM Agents in Long-Context Software Engineering*. arXiv, November 2025. arXiv:2511.13998.

[3] PRDBench: *Automatically Benchmarking LLM Code Agents through Agent-Driven Annotation and Evaluation*. arXiv, October 2025. arXiv:2410.XXXXX. To appear at AAMAS 2026.

[4] ETH Zurich / University of Bern (2026). *Context File Provision in LLM Coding Agents: When More Context Is Less.* arXiv:2602.11988. (LLM-generated AGENTS.md degrades task success −3%; increases cost +20%.)

[5] Chen, M., Tworek, J., Jun, H., et al. (2021). *Evaluating Large Language Models Trained on Code (HumanEval).* arXiv:2107.03374.

[6] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020. arXiv:2005.11401.

[7] Packer, C., et al. (2023). *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560.

[8] Knuth, D. E. (1984). Literate Programming. *The Computer Journal*, 27(2), 97–111.

[9] GitHub Copilot. (2024). *Custom instructions for GitHub Copilot.* docs.github.com.

[10] Anthropic. (2024). *Claude's memory: CLAUDE.md.* docs.anthropic.com.

[11] Tongyi Lingma Team (2025). *Repository-Level Code Understanding via Knowledge Graph Traversal.* Alibaba DAMO Academy preprint.

[12] Karpathy, A. (2025). *LLM-wiki: compile a codebase into a navigable knowledge graph.* GitHub Gist. gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

[13] Corpora, F. (2026). *CodeDNA v0.7 — original preprint.* Zenodo. doi:10.5281/zenodo.19158336.